FLEETIVE

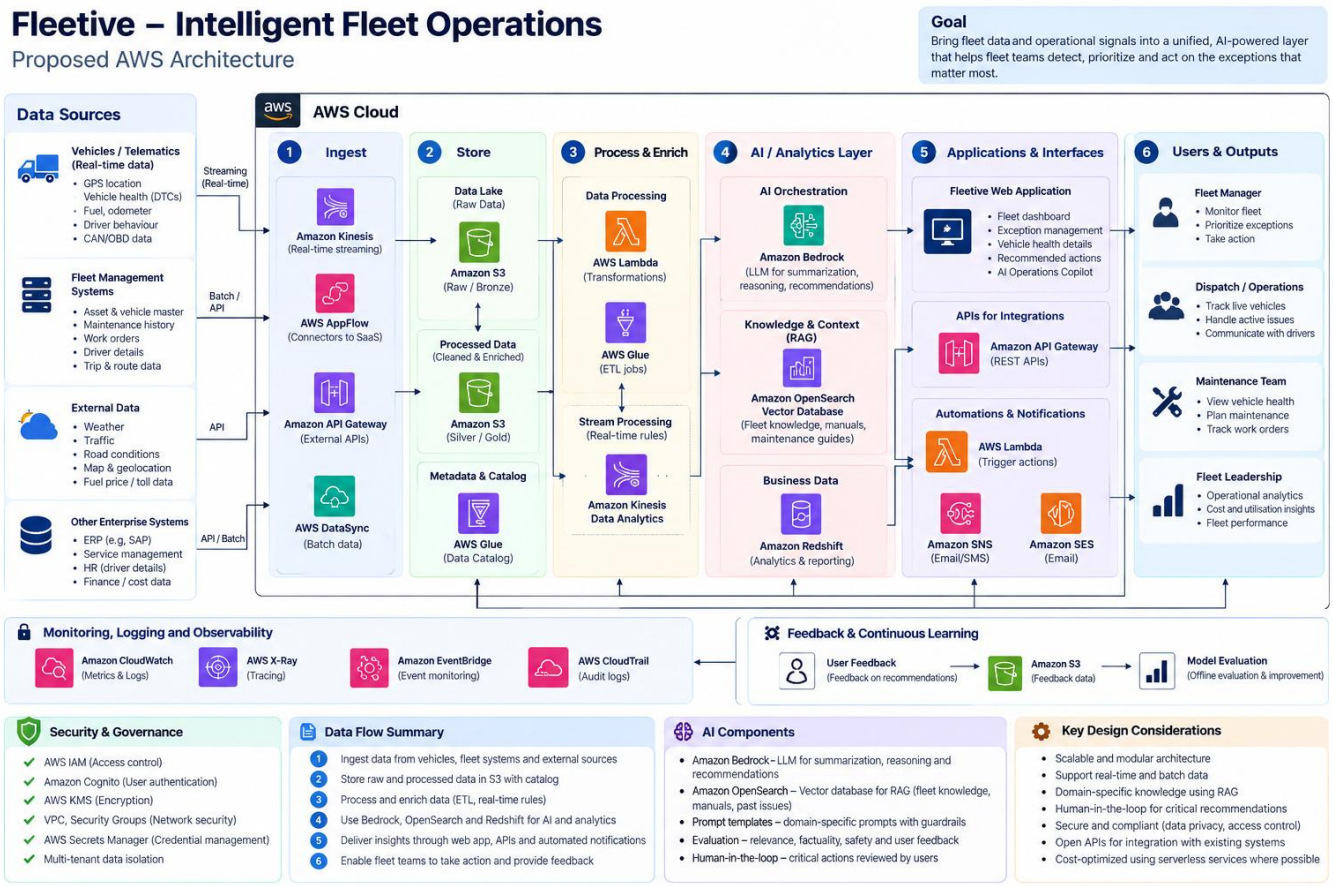
Proposed AWS Architecture

A product-led architecture proposal for an AI-assisted fleet exception copilot

How I approached this: I started with the operational decision a fleet team needs to make, then worked backwards into the data flows, services, controls, and feedback loops needed to support it. I am presenting this as a proposed architecture for the Fleetive portfolio concept—not as a deployed AWS environment.

1. Architecture Diagram

The diagram below shows my proposed end-to-end design: fleet and enterprise data enter through ingestion services, are stored and processed, enriched with AI and operational context, and surfaced through Fleetive interfaces and integrations. Feedback and monitoring support continuous evaluation.



Architecture note: service selection is illustrative and should be validated against source-system interfaces, latency requirements, expected event volume, data residency, and the security model. The diagram is a design artifact.

2. My Architecture Goals

- Bring vehicle, maintenance, dispatch, route, and external signals into a consistent operational view.

- Support both streaming events and scheduled batch integrations without forcing every source into one ingestion pattern.
- Separate deterministic rules and severity decisions from AI-generated summaries or recommendations.
- Make recommendations explainable, traceable to evidence, and reviewable by an authorized person before consequential action.
- Start with a small pilot architecture and introduce additional services only when scale, latency, or operational needs justify them.

3. Proposed Architecture by Layer

A. Data sources

Vehicle telematics may provide location, diagnostic trouble codes, odometer, fuel, and driver-behaviour signals. Fleet management systems may provide vehicle and asset masters, maintenance history, work orders, driver details, and trip data. External feeds can include weather, traffic, road conditions, and geolocation. ERP and service systems may contribute cost and repair context. I would validate data ownership, availability, and refresh frequency for each source before finalizing integrations.

B. Ingestion and integration

For REST-based sources, I would expose controlled integration endpoints through Amazon API Gateway. AWS AppFlow may help with supported SaaS connectors, and AWS DataSync may be useful for scheduled file movement where the source and transfer pattern fit. For real-time event streams, Amazon Kinesis is a candidate. I would use AWS IoT Core only if the vehicle or gateway integration genuinely requires MQTT/device connectivity; I would not add it by default.

C. Storage and catalog

Amazon S3 can hold raw source data and curated datasets, organized by source, date, and schema/version. I would keep raw records immutable where feasible for audit and replay, and use separate curated zones for normalized data. AWS Glue Data Catalog can describe datasets for discovery and analytics. For operational state that needs transactional updates and low-latency queries, I would evaluate Amazon RDS for PostgreSQL rather than treating the data lake as the application database.

D. Processing and event routing

AWS Lambda can handle lightweight transformations and event-driven tasks. Amazon EventBridge can route domain events, while Amazon SQS can buffer work and absorb traffic spikes; a dead-letter queue would capture messages that repeatedly fail. AWS Glue can support larger ETL jobs. I would keep processing idempotent, version schemas, and define replay and duplicate-handling behaviour.

E. AI and knowledge retrieval

Amazon Bedrock is a candidate for summarizing an exception, explaining supporting evidence in plain language, and drafting recommended next steps. For retrieval-augmented generation, I would index approved fleet manuals, operating procedures, and relevant historical cases in a retrieval layer such as Amazon OpenSearch Service with vector search, subject to a validated retrieval design. The model should receive only the minimum authorized context needed for the task.

F. Application and APIs

The Fleetive web application would present fleet health, ranked exceptions, issue details, evidence, and recommended actions. Amazon API Gateway can front application APIs, with Lambda or container services such as Amazon ECS on AWS Fargate hosting business logic depending on workload and deployment needs. Amazon Cognito or enterprise

federation can provide authentication. I would separate read-only insights from action-taking APIs and enforce authorization on the server.

G. Analytics, notifications, and feedback

Amazon Redshift is an option for governed analytical reporting when reporting needs justify a warehouse; for an early pilot, S3 with Athena may be sufficient. Amazon SNS can deliver supported notifications, and event-driven Lambda functions can initiate approved workflows. User feedback, reviewer corrections, and outcome labels should be stored separately and used in a controlled evaluation process—not fed automatically into production model behaviour without review.

H. Security, operations, and governance

I would apply least-privilege IAM, encryption with AWS KMS, secrets management, network segmentation, and tenant-level access controls. Amazon CloudWatch can provide metrics and logs, AWS X-Ray can trace supported distributed requests, and AWS CloudTrail can record account/API activity. Sensitive driver or location data needs explicit retention, access, purpose-limitation, and audit rules.

4. Core Data Flow

1. Ingest: collect telemetry and operational records through streaming, API, or batch pathways appropriate to each source.
2. Normalize: map source-specific records to a canonical event schema, validate required fields, attach timestamps and source identifiers, and reject or quarantine invalid records.
3. Persist: retain raw data in S3 and store current operational entities and workflow state in a transactional store where required.
4. Enrich: join vehicle, maintenance, route, and policy context; apply deterministic rules for severity, eligibility, and escalation.
5. Assist: use retrieval and an LLM to summarize the issue and draft evidence-grounded recommendations. The model does not independently authorize high-impact actions.
6. Present and act: show the ranked exception, evidence, uncertainty, and recommended next step. Require a permitted human reviewer for consequential actions.
7. Learn: capture whether the recommendation was accepted, edited, dismissed, or resolved, then use reviewed labels for offline evaluation and product improvements.

5. AI Design and Evaluation

I would not use an LLM as the source of truth for vehicle state or safety-critical severity. Deterministic rules and source data should establish the event facts and baseline severity; the AI layer should help interpret the context and communicate it clearly. Each recommendation should cite the source signals or retrieved procedure used, state when evidence is missing, and abstain rather than invent a diagnosis.

- Evaluate retrieval relevance and coverage against a curated set of fleet questions and documents.
- Measure factual consistency, evidence citation quality, recommendation usefulness, and unsupported-claim rate.
- Test prompt injection, cross-tenant retrieval leakage, stale documentation, missing fields, and conflicting signals.
- Use human review for high-impact recommendations and maintain an auditable record of model version, prompt/template version, retrieved context identifiers, and reviewer decision.
- Use offline evaluation and a controlled pilot before broad rollout; do not claim production accuracy or savings without measured results.

6. Reliability and Failure Handling

- Use retries with backoff for transient failures and dead-letter queues for messages that cannot be processed.
- Make event handlers idempotent and track event IDs to reduce duplicate processing.
- Define freshness indicators so users can see when telemetry or a source integration is delayed.
- Provide graceful degradation: when the model or retrieval service is unavailable, preserve deterministic alerts and show that AI assistance is unavailable.
- Set service-level targets only after establishing pilot baselines for latency, availability, event volume, and recovery needs.

7. Prototype-to-Production Plan

Phase 1 — Prototype

Use synthetic or appropriately anonymized sample data to validate the workflow, exception cards, and AI response format. Avoid implying that the current prototype is connected to live fleet systems.

Phase 2 — Controlled pilot

Integrate a small number of approved sources, define access and retention controls, select a narrow exception category, and run AI recommendations in shadow or human-reviewed mode.

Phase 3 — Production readiness

Complete threat modelling, load and resilience testing, data-quality checks, model evaluation, operational runbooks, alerting, and support ownership.

Phase 4 — Scale

Expand sources and exception types only after adoption, reliability, recommendation quality, and operational outcomes meet agreed thresholds.

8. Key Trade-offs and Decisions I Would Validate

- Streaming versus batch: use streaming only where freshness changes an operational decision; batch may be simpler and cheaper for slower-moving records.
- Lambda versus containers: use Lambda for event-driven, bounded work; evaluate ECS/Fargate for long-running services or more complex runtime requirements.
- OpenSearch vector retrieval versus another managed knowledge store: decide after testing retrieval quality, metadata filtering, operational burden, and cost.
- Redshift versus Athena: begin with the simplest analytics option that meets latency and concurrency needs; add a warehouse only when justified.
- IoT Core: include only if the actual source protocol and device architecture require it.
- Autonomy: keep actions advisory or human-approved until risk, permissions, rollback, and audit requirements are explicitly validated.

9. Assumptions, Risks, and Open Questions

- Assumption: source systems can expose data through approved APIs, exports, or event streams; this must be confirmed with system owners.

- Risk: inconsistent vehicle identifiers and timestamps can create incorrect joins or duplicate exceptions. Mitigation: canonical IDs, data-quality checks, and reconciliation.
- Risk: sensitive driver/location information may be over-collected. Mitigation: data minimization, access controls, retention policies, and privacy review.
- Risk: an LLM may produce plausible but unsupported recommendations. Mitigation: retrieval grounding, output schemas, citations, evaluation, abstention, and human review.
- Open question: what is the required freshness for each exception type, and which decisions genuinely need real-time data?
- Open question: which system is authoritative for vehicle identity, maintenance status, work orders, and resolution outcomes?
- Open question: what are the pilot fleet size, expected event rate, regions, retention period, availability target, and budget?

10. Description

I designed this as a proposed AWS architecture for Fleetive, connecting the product workflow to ingestion, processing, data storage, AI-assisted interpretation, integrations, and governance. My focus was to make the AI layer useful without letting it become the source of truth: deterministic signals establish the facts, AI helps explain and prioritize them, and people retain control over consequential actions. The design is a portfolio proposal and would need to be validated with source-system owners, security stakeholders, and pilot users before implementation.